

Práctica guiada:

APIs: Retrofit 2 con Recycler

Introducción

Una API es un intermediario entre una base de datos y nuestra app. El consumo de APIs permite obtener información de un cliente externo.

El uso de estas APIs es una de las tareas mas necesarias para tener una app útil.

Como norma general existen cuatro tipos de peticiones:

- **Get:** Son las peticiones más sencillas, solo nos devuelven información. Si necesitamos pasarle un parámetro a la petición será a través de la url.
- **Post:** Similar a Get pero los parámetros no se pasan por url, por lo que es más seguro para mandar información.
- **Put:** Se suele usar para crear la entidad,
- **Delete:** Nos permitiría borrar los registros de la base de datos.

La información devuelta suele tener los formatos XML o JSON. Siendo esta segunda la más habitual.

Json es un estándar para el traspaso de información con un formato de texto simple y es el acrónimo de JavaScript Object Notation

Ejemplo:

```
{
  "nombre": "Daniel",
  "email": "daniel.amigueta@murciaeduca.es",
  "asignaturas": [
    {
      "nombre": "PMDM",
      "curso": 2
    },
    {
      "nombre": "programacion",
      "curso": 1
    }
  ]
}
```

El formato JSON empieza y termina con llaves con diferentes clave-valor. En nuestro ejemplo, la clave asignaturas contiene a su vez una lista de asignaturas con diferentes clave-valor. Esta lista empieza y termina con corchetes y se separan por comas.

Práctica guiada

En esta práctica vamos a realizar un buscador de autores de libros que utilizará la API pública de OpenLibrary y mostrará los datos en un RecyclerView.

<https://openlibrary.org/developers/api>

Para esta práctica usaremos Authors API



Dado que nuestra app obtendrá los datos desde internet necesitamos añadir el permiso de en nuestro manifest:

```
<uses-permission android:name="android.permission.INTERNET"/>
```

El siguiente paso, como viene siendo habitual, es añadir la librerías necesarias en el gradle del módulo de la app.

Las librerías son:

```
implementation ("com.squareup.retrofit2:retrofit:2.11.0")
implementation ("com.squareup.retrofit2:converter-gson:2.11.0")
```

Donde:

- Retrofit 2: Librería encargada del consumo de las API.
- Retrofit 2 Converter Gson: Complemento de Restrofit para simplificarás el proceso de pasar un JSON a una Data Class.

Una vez configuradas nuestras librerías, vamos a modificar nuestro layout principal al que le añadiremos un Recyclerview y un Searchview. El componente de búsqueda es un elemento muy sencillo de implementar para obtener un buscador de una forma muy rápida y sencilla.

activity_main.xml

```

<androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:app="http://schemas.android.com/apk/res-auto"
xmlns:tools="http://schemas.android.com/tools"
android:id="@+id/viewRoot"
android:layout_width="match_parent"
android:layout_height="match_parent"
tools:context=".MainActivity">

    <androidx.appcompat.widget.SearchView
        android:id="@+id/svAutores"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        app:layout_constraintVertical_bias="0"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintLeft_toLeftOf="parent"
        app:layout_constraintRight_toRightOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

    <androidx.recyclerview.widget.RecyclerView
        android:id="@+id/rvAutores"
        android:layout_width="0dp"
        android:layout_height="0dp"
        app:layout_constraintTop_toBottomOf="@+id/svAutores"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintBottom_toBottomOf="parent"/>

</androidx.constraintlayout.widget.ConstraintLayout>

```

Ahora configuramos nuestro fila_autores que solo contendrá un texview con el nombre del autor

```

<androidx.cardview.widget.CardView
xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:app="http://schemas.android.com/apk/res-auto"
app:cardCornerRadius="16dp"
android:layout_margin="16dp"
android:layout_height="wrap_content">
    <TextView
        android:id="@+id/tvAutor"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:padding="10dp"
        android:text="nombre_autor"
        android:textSize="30sp"
        android:textStyle="bold"
    />
</androidx.cardview.widget.CardView>

```

Las peticiones API nos devolverán un JSON que tendremos que convertir en una Data Class.

Si accedemos con un navegador a nuestra API obtendremos un ejemplo de la estructura de la respuesta:

<https://openlibrary.org/dev/docs/api/authors>

```
{
  numFound: 1,
  start: 0,
  numFoundExact: true,
  docs: [
    {
      key: "OL23919A",
      text: [...],
      type: "author",
      name: "J. K. Rowling",
      alternate_names: [...],
      birth_date: "31 July 1965",
      top_work: "Harry Potter and the Philosopher's Stone",
      work_count: 162,
      top_subjects: [...],
      _version_: 1702166143068799000
    },
  ]
}
```

Este objeto recibe numFound, start, numFoundExact y una lista de autores. A nosotros, de momento, nos interesa el nombre de cada item de la lista.

Por tanto, vamos a crear una nueva clase que lo llamaremos AuthorsResponse en el cual añadimos el parámetro numFound y una lista de Autores

```
class AuthorsResponse (var numFound: Int, var docs: List<Autores>){
}
```

Como podréis apreciar, el nombre de los campos es exactamente igual que el JSON, ya que es obligatorio para que las librerías hagan la conversión correctamente.

Si el nombre docs no fuera de vuestro agrado, podemos utilizar la anotación **SerializedName**

```
data class AuthorsResponse (var numFound: Int, @SerializedName("docs") var autores:
List<Autores>){
}
```

Dado que en docs tenemos una lista de objetos, vamos a crear una clase de tipo Autores para recoger esta información.

```
class Autores (var key: String, var birth_date: String, var name: String){
}
```

APIService

Ahora crearemos una interfaz que defina el tipo de consumo de API y lo que nos devolverá.

En esta interfaz solo definimos el método que usará Retrofit y su configuración

```
interface APIService {  
    @GET  
    suspend fun getAutores(@Url url:String): Response<AuthorsResponse>  
}
```

Instancia Retrofit 2

Para usar Retrofit necesitamos crear una instancia de Retrofit, que contendrá la base de la url de la API y se encargará de convertir el JSON a AuthorsResponse.

Esta función se creará en nuestro MainActivity

```
private fun getRetrofit(): Retrofit {  
    return Retrofit.Builder()  
        .baseUrl("https://openlibrary.org/")  
        .addConverterFactory(GsonConverterFactory.create())  
        .build()  
}
```

Ahora creamos la función que recibirá un String como parámetro. Esta función será la llamada por el buscador y devolverá los resultados.

```
private fun busquedaAutores(query:String){  
    CoroutineScope(Dispatchers.IO).launch {  
    }  
}
```

Como vemos, lo primero que hacemos es llamar a CoroutineScope para crear un hilo asíncrono.

El siguiente paso es crear una variable para llamar al método que nos devuelve Retrofit y a esa instancia de Retrofit tendremos que llamar a la función **create()**, la cual recibe la interfaz que queramos, en nuestro caso getAutores

```
private fun busquedaAutores(query:String){  
    lifecycleScope.launch(Dispatchers.IO) {  
        val call = getRetrofit().create(APIService::class.java).getAutores("/search/authors.json?q=$query")  
    }  
}
```

Ahora la variable call contiene un Response<AuthorsResponse>. Con la llamada a call.isSuccessful sabremos si la llamada ha sido correcta

```
private fun busquedaAutores(query:String){
    lifecycleScope.launch(Dispatchers.IO) {
        val call = getRetrofit().create(APIservice::class.java).getAutores("/search/authors.json?q=$query")
        val autoresAPI = call.body()
        if(call.isSuccessful){
            //cargar datos en RecyclerView

        }else{
            //mostrar error
        }
    }
}
```

Vamos a volver con nuestro recyclerView ya que nos faltaba implementar el ViewHolder y el adaptador.

```
class AutoresViewHolder (view: View): RecyclerView.ViewHolder(view) {
    var nombre: TextView
    init{
        nombre=view.findViewById(R.id.tvAutor)
    }
}
```

```
class AdaptadorAutores : RecyclerView.Adapter<AutoresViewHolder>{
    private var aut: List<Autores>
    constructor(){
        aut=ArrayList()
    }

    override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): AutoresViewHolder {
        val inflater = LayoutInflater.from(parent.context)
        return AutoresViewHolder(inflater.inflate(R.layout.fila_autores, parent, false))
    }
    override fun getItemCount(): Int = aut.size
    override fun onBindViewHolder(holder: AutoresViewHolder, position: Int) {
        val item = aut[position]
        holder.nombre.text = item.name
    }
}
```

Configuración Final

En nuestro MainActivity, en la parte superior de la clase, creamos nuestro adaptador y resto de componentes

```
private lateinit var recy: RecyclerView
val adaptadorRecyclerView = AdaptadorAutores()
private lateinit var buscador: SearchView
```

Y bindeamos en onCreate

```
//bind
buscador=findViewById(R.id.svAutores)
recy = findViewById(R.id.rvAutores)
recy.layoutManager = LinearLayoutManager(applicationContext,
    LinearLayoutManager.VERTICAL, false)
recy.adapter = adaptadorRecyclerView
```

Llegados a este punto ya tenemos el RecyclerView creado, aunque todavía no añadimos los datos.

Por tanto, vamos a volver al método busquedaAutores()

```
if(call.isSuccessful){
    val autores = autoresAPI?.autores?:emptyList()
    withContext(Dispatchers.Main) {
        adaptadorRecyclerView.changelist(autores)
        adaptadorRecyclerView.notifyDataSetChanged()
    }
}else{
    Toast.makeText(applicationContext, "error API", Toast.LENGTH_SHORT).show()
}
```

Donde changelist() unicamente cambia un List<Autores> por uno nuevo

```
fun changelist(autores: List<Autores>) {
    aut=autores
}
```

Configuración buscador

Por último, solo nos queda implementar los listeners del SearchView.

En la primera línea del **MainActivity**, a continuación de **AppCompatActivity()** añadiremos el *listener* SearchView.OnQueryTextListener.

```
class MainActivity : AppCompatActivity(), SearchView.OnQueryTextListener{
```

Automáticamente nos sale un error en la clase ya que debemos de implementar dos métodos.

```
override fun onQueryTextSubmit(query: String?): Boolean {  
    TODO("Not yet implemented")  
}  
  
override fun onQueryTextChange(newText: String?): Boolean {  
    TODO("Not yet implemented")  
}
```

El método onQueryTextChange avisa de cada carácter que se añade al buscador. En nuestro caso no necesitamos este método por lo que devolvemos true y no hacemos nada más

```
override fun onQueryTextChange(newText: String?): Boolean {  
    return true  
}
```

El método onQueryTextSubmit será llamado cuando el suario pulse enter al terminar de buscar.

```
override fun onQueryTextSubmit(query: String?): Boolean {  
    if(!query.isNullOrEmpty()){  
        busquedaAutores(query.lowercase())  
    }  
    val imm = getSystemService(INPUT_METHOD_SERVICE) as InputMethodManager  
    imm.hideSoftInputFromWindow(this.buscador.windowToken, 0)  
    return true  
}
```

Y por último nos queda asociar estos listeners a nuestro buscador, por lo que si volvemos a nuestro bindeo del buscador solo debemos de añadir la siguiente linea.

```
buscador.setOnQueryTextListener(this)
```